

Adapter Design Pattern

Category: Structural | **GoF Reference:** Gang of Four — Design Patterns (1994), p. 139

Table of Contents

1. [Intent](#)
 2. [When to Use](#)
 3. [Structure — ASCII UML Diagram](#)
 4. [Package Structure](#)
 5. [Line-by-Line Explanation of Every File](#)
 6. [Printer.java — Target Interface](#)
 7. [LegacyPrinter.java — Adaptee](#)
 8. [PrinterAdapter.java — Adapter](#)
 9. [AdapterDesignPattern.java — Driver / Client](#)
 10. [Execution Flow](#)
 11. [Real-World Use Cases](#)
 12. [Class Adapter vs Object Adapter](#)
 13. [Key Design Decisions](#)
 14. [Output](#)
 15. [Summary](#)
-

Intent

The Adapter pattern converts the interface of one class into another interface that the client expects. It allows classes with incompatible interfaces to work together without modifying their source code. The pattern acts as a structural bridge: a wrapper object — the Adapter — implements the interface the client knows about (the Target), holds a reference to the object whose interface is incompatible (the Adaptee), and internally translates every call from the Target's contract into the equivalent call on the Adaptee. The result is that the client is completely unaware of the Adaptee's existence; from the client's perspective it is simply talking to a standard `Printer`, not a `LegacyPrinter` with a different method name. This makes the pattern especially valuable when integrating existing or third-party code that cannot be modified.

When to Use

Use the Adapter pattern in any of these situations:

Legacy integration: You have a working class that predates your current interface contract. Rewriting it is risky, expensive, or impossible (no source code access). The Adapter lets you reuse the legacy class without touching it.

Third-party libraries: An external library ships a class whose method signatures differ from your application's interface. Writing an adapter avoids coupling your entire codebase to the third-party API surface; if the library changes, only the adapter changes.

Interface mismatch during team collaboration: Two teams independently develop two components that must cooperate. They converged on slightly different method names or signatures. An Adapter reconciles the difference without a breaking change on either side.

Incremental migration: You are replacing a legacy system piece by piece. The Adapter lets the new system speak to the old system through a translation layer while the migration is in progress, removing the need for a big-bang cutover.

Testing and mocking: An Adapter wrapping a heavyweight external resource (database, network socket, printer driver) lets you swap in a lightweight test double that satisfies the same Target interface.

Structure — ASCII UML Diagram

```
Client | | uses v +-----+ +-----+ | <<interface>> | |
Adaptee | | Target | | (LegacyPrinter) | |-----| |-----| | +
print() | | + printDocument() | +-----+ +-----+ ^ ^ |
implements | has-a (composition) | | +-----+ | | Adapter
|-----+ | (PrinterAdapter) | |-----| | - legacyPrinter |
|-----| | + print() | ---delegates-to--> legacyPrinter.printDocument()
+-----+
```

Reading the diagram:

- The **Client** (`AdapterDesignPattern.main`) depends only on the `Target` interface (`Printer`). It never references `LegacyPrinter` directly.
- The **Target** (`Printer`) declares the interface the client was written against — a single `print()` method.

- The **Adaptee** (`LegacyPrinter`) is the existing class with an incompatible method name (`printDocument()`).
- The **Adapter** (`PrinterAdapter`) simultaneously satisfies the Target interface (via `implements Printer`) and holds a reference to the Adaptee (via composition). Its `print()` method delegates the work to `legacyPrinter.printDocument()`.

Package Structure

```
com.design.patterns.adapter ■■■ AdapterDesignPattern.java (driver / client – root package) ■ ■■■ target/ ■ ■■■ Printer.java (Target interface) ■ ■■■ adaptee/ ■ ■■■ LegacyPrinter.java (Adaptee – the class being adapted) ■ ■■■ adapter/ ■■■ PrinterAdapter.java (Adapter – the bridge)
```

Why three separate packages?

`target/` — the contract package.

The `Printer` interface represents the abstraction the application is built against. Isolating it in its own package signals that this is the stable API surface. New Adapters, mock implementations, and production implementations all live in their own packages and depend on this one. If `Printer` ever needs a second implementation (e.g., a `NetworkPrinter` adapter), the `target/` package remains untouched.

`adaptee/` — the foreign/legacy code package.

`LegacyPrinter` lives in `adaptee/` to make it explicit that this class is the thing being adapted — it owns its own behavior and has no knowledge of `Printer`. In a real project this package might be replaced by an import from a third-party jar; keeping it separate mirrors that reality and avoids accidental coupling.

`adapter/` — the translation layer package.

`PrinterAdapter` is the only class that is allowed to import from both `target/` and `adaptee/`. Keeping it in its own package makes the dependency direction visible in the package structure itself: `adapter/` depends on `target/` and `adaptee/`; nothing else does.

Root package — the client.

`AdapterDesignPattern` is the driver. It lives in the root package so that it can import from all sub-packages. In a production application the "client" would be a service class or a Spring `@Component`; the driver here simply stands in for that role.

Line-by-Line Explanation of Every File

1. `Printer.java` — Target Interface

```
package com.design.patterns.adapter.target;
```

What it does: Declares the Java package that owns this type.

Why it is written this way: Placing the interface in the `target` sub-package separates the "what we expect" contract from all implementation details. Any code that wants to print — the client, tests, alternative adapters — imports from this package alone, which enforces the Dependency Inversion Principle: depend on abstractions, not concretions.

```
public interface Printer {
```

What it does: Declares a public Java interface named `Printer`.

Why it is written this way: An `interface` rather than an abstract class is used because the Target defines only a contract — zero shared state, zero shared behaviour. The `public` modifier is required so that classes in other packages (the adapter, the client) can implement and reference it. Using an interface also allows `PrinterAdapter` to implement `Printer` while independently extending any other class if needed (Java does not support multiple class inheritance but does support multiple interface implementation).

```
void print();
```

What it does: Declares a single abstract method `print()` with no parameters and no return value.

Why it is written this way: This is the method the client calls. Its name is the "modern" or "expected" name that the rest of the system has been written against. The Adaptee's equivalent method is called `printDocument()` — a different name that the client does not know. The Adapter's sole job is to map `print()` → `printDocument()`. Keeping the interface minimal (one method) follows the Interface Segregation Principle: the client depends only on what it uses.

```
}
```

Closes the interface declaration.

2. `LegacyPrinter.java` — Adaptee

```
package com.design.patterns.adapter.adaptee;
```

What it does: Declares the package for the legacy (adaptee) code.

Why it is written this way: Isolating legacy code in its own `adaptee` package communicates its role to every reader of the codebase. It also mirrors how real-world legacy or third-party code lands in a project — as an external dependency with its own namespace that you do not control.

```
public class LegacyPrinter {
```

What it does: Declares the `LegacyPrinter` concrete class.

Why it is written this way: It is a `class`, not an interface, because it already has behaviour. It does not implement `Printer` — that is the whole problem the Adapter solves. In a real system this class might be compiled into a `.jar` you cannot modify; the pattern works identically whether you have the source or not.

```
public void printDocument() {
```

What it does: Declares a public method `printDocument()` — the Adaptee's own method signature.

Why it is written this way: The method name `printDocument()` differs from `print()`, which is the root cause of the incompatibility. The method is `public` so the `PrinterAdapter` in a different package can call it. There is no `@Override` because `LegacyPrinter` does not implement any interface.

```
System.out.println("Legacy Printer");
```

What it does: Prints the string `"Legacy Printer"` to standard output.

Why it is written this way: This line represents the real work the legacy system performs — in a production context it might drive a hardware printer, write to a print queue, or call a vendor API. The `System.out.println` here is a stand-in that proves the Adaptee's logic is actually being invoked when the Adapter delegates to it.

```
} }
```

Closes the method and the class declaration.

3. `PrinterAdapter.java` — Adapter

```
package com.design.patterns.adapter.adapter;
```

What it does: Declares the package for the adapter layer.

Why it is written this way: The `adapter` sub-package is the only package in this module that imports from both `target` and `adaptee`. Housing all translation logic here creates a clear seam: if the legacy API ever changes, only files in this package need to change.

```
import com.design.patterns.adapter.adaptee.LegacyPrinter;
```

What it does: Brings the `LegacyPrinter` class into scope.

Why it is written this way: The Adapter must hold a reference to the Adaptee, so it must know the Adaptee's type. This is the only file outside the `adaptee` package that imports from it. If the Adaptee were replaced by a different legacy class, only this import line and the constructor type would change.

```
import com.design.patterns.adapter.target.Printer;
```

What it does: Brings the `Printer` Target interface into scope.

Why it is written this way: The Adapter implements the Target, so it must reference it. The fact that `PrinterAdapter` imports from both `target` and `adaptee` packages is not a design flaw — it is the Adapter's explicit, intentional role.

```
public class PrinterAdapter implements Printer {
```

What it does: Declares `PrinterAdapter` as a concrete class that fulfils the `Printer` interface contract.

Why it is written this way: `implements Printer` is what makes the Adapter substitutable for any `Printer` reference throughout the codebase. The client can write `Printer p = new PrinterAdapter(...)` and never know what is on the other side. The class is `public` so the client (in the root package) can instantiate it. The class is not declared `final` because a subclass could theoretically extend it to adapt a further-evolved legacy class.

```
private final LegacyPrinter legacyPrinter;
```

What it does: Declares an instance field of type `LegacyPrinter`, marked `private` and `final`.

Why it is written this way:

- `private` — the field is an implementation detail. Nothing outside `PrinterAdapter` should reach through the adapter to manipulate the Adaptee directly. Hiding it prevents accidental coupling.

- `final` — once the adapter is constructed, the Adaptee it wraps never changes. `final` enforces this invariant at compile time, makes the object effectively immutable with respect to its collaborator, and is safe to use in multi-threaded environments without additional synchronization.
- Type `LegacyPrinter` — the field holds the concrete Adaptee. In more flexible designs the field type might be an interface or abstract class so that the Adapter can work with any of several legacy implementations; for this focused example, the concrete type is sufficient.

```
public PrinterAdapter(LegacyPrinter legacyPrinter) {
```

What it does: Declares a public constructor that accepts a `LegacyPrinter` instance.

Why it is written this way: Constructor injection is the idiomatic way to supply a collaborator in plain Java and in dependency injection frameworks (Spring, Guice, CDI). The Adaptee is a required dependency — the Adapter cannot function without it — so it is passed at construction time rather than via a setter, making it impossible to create a partially initialised Adapter. Accepting `LegacyPrinter` as a parameter (rather than creating it inside the constructor) is the Dependency Inversion Principle in action: the Adapter does not decide which `LegacyPrinter` to use; the caller does.

```
this.legacyPrinter = legacyPrinter;
```

What it does: Assigns the constructor parameter to the instance field.

Why it is written this way: The `this.` qualifier disambiguates the instance field from the constructor parameter, which share the same name. The assignment completes the initialisation of the `final` field, satisfying the Java compiler's definite-assignment rule for `final` fields.

```
}
```

Closes the constructor.

```
@Override public void print() {
```

What it does: Implements the `print()` method declared in the `Printer` interface.

Why it is written this way: `@Override` is a compile-time annotation that instructs the compiler to verify that this method genuinely overrides or implements a method from a parent type. If someone accidentally renamed `print()` to `Print()` in the interface, the compiler would immediately flag the mismatch here. This is the Adapter's core method — the single point of translation between the Target's contract and the Adaptee's API.

```
legacyPrinter.printDocument();
```

What it does: Calls the Adaptee's `printDocument()` method on the stored `legacyPrinter` reference.

Why it is written this way: This one line is the entire translation. The client calls `print()`; the Adapter silently reroutes that call to `printDocument()`. In more complex scenarios this line might also transform parameters, convert return types, handle exceptions, or orchestrate multiple Adaptee calls to satisfy one Target call. Here the mapping is 1:1 with only a name change, which keeps the example focused.

```
} }
```

Closes the method and the class declaration.

4. `AdapterDesignPattern.java` — Driver / Client

```
package com.design.patterns.adapter;
```

What it does: Places the driver class in the root `adapter` package.

Why it is written this way: The root package sits above all sub-packages, so it can import from `target`, `adaptee`, and `adapter` without circular dependency. In a real application the client would not normally import from `adaptee` at all — it would receive the `Printer` via dependency injection and never construct a `LegacyPrinter` directly.

```
import com.design.patterns.adapter.adaptee.LegacyPrinter;
```

What it does: Imports `LegacyPrinter` so it can be instantiated in `main`.

Why it is written this way: The driver must create a `LegacyPrinter` instance to pass into the `PrinterAdapter` constructor. This is the composition root — the one place in an application where concrete types are assembled together. In a Spring application a `@Bean` factory method would play this role, keeping the rest of the codebase free from knowing about `LegacyPrinter`.

```
import com.design.patterns.adapter.adapter.PrinterAdapter;
```

What it does: Imports `PrinterAdapter` so it can be instantiated in `main`.

Why it is written this way: The driver is the only code that knows the concrete Adapter type. All subsequent usage goes through the `Printer` interface variable.


```
import com.design.patterns.adapter.target.Printer;
```

What it does: Imports the `Printer` Target interface.

Why it is written this way: The variable `printer` is declared as type `Printer`, not `PrinterAdapter`. This is the critical design point: after construction, the rest of the code deals exclusively with the interface. Swapping to a completely different `Printer` implementation (e.g., a `NetworkPrinterAdapter`) would require changing only the two lines inside `main` that do construction — nothing else.

```
public class AdapterDesignPattern {
```

What it does: Declares the public driver class.

Why it is written this way: Named after the pattern to make the module's purpose self-documenting. In a real project this would be replaced by an application entry point (`@SpringBootApplication`, a CLI runner, etc.).

```
public static void main(String[] args) {
```

What it does: Declares the JVM entry point.

Why it is written this way: `public static void main(String[] args)` is the mandated signature for a Java application entry point. `static` means no instance of `AdapterDesignPattern` is needed to run the program. `String[] args` allows command-line arguments to be passed, though this example does not use them.

```
System.out.println("Adapter Design Pattern");
```

What it does: Prints a header line identifying the running pattern.

Why it is written this way: This line is purely presentational — it labels the console output so that when multiple pattern demos are run in sequence, the reader knows which pattern produced which output.

```
Printer printer = new PrinterAdapter(new LegacyPrinter());
```

What it does: Creates a `LegacyPrinter` instance, wraps it in a `PrinterAdapter`, and stores the result in a variable declared as type `Printer`.

Why it is written this way: Three things happen here:

1. `new LegacyPrinter()` — instantiates the Adaptee. This is the object that carries the real behaviour.
2. `new PrinterAdapter(...)` — wraps the Adaptee in the Adapter, passing it via constructor injection.
3. `Printer printer = ...` — declares the variable as the interface type, not the concrete type. From this line onward the rest of the code knows only that `printer` is a `Printer`. The Adapter and the `LegacyPrinter` are hidden behind the interface.

```
printer.print();
```

What it does: Invokes the `print()` method on the `Printer` interface reference.

Why it is written this way: The client calls the method it was designed to call — `print()`. At runtime, dynamic dispatch routes this call to `PrinterAdapter.print()`, which in turn calls `LegacyPrinter.printDocument()`. The client code is completely isolated from that routing; it simply asks a `Printer` to print.

```
} }
```

Closes the `main` method and the class declaration.

Execution Flow

The following numbered trace walks through every method call from JVM startup to the final printed line.

```
1. JVM loads AdapterDesignPattern and invokes main(String[] args) ■ 2.
System.out.println("Adapter Design Pattern") ■ → Prints: "Adapter Design Pattern" ■ 3.
new LegacyPrinter() ■ → Java allocates a LegacyPrinter object on the heap. ■ → No
constructor body defined; the default no-arg constructor runs silently. ■ 4. new
PrinterAdapter( <LegacyPrinter instance> ) ■ → Java allocates a PrinterAdapter object on
the heap. ■ → PrinterAdapter(LegacyPrinter legacyPrinter) executes: ■ this.legacyPrinter
= legacyPrinter; ← stores the reference ■ 5. Printer printer = <PrinterAdapter instance>
■ → The variable 'printer' holds a reference to the PrinterAdapter, ■ but its declared
type is Printer (the interface). ■ 6. printer.print() ■ → Dynamic dispatch: the JVM looks
up the runtime type of 'printer' ■ (which is PrinterAdapter) and calls
PrinterAdapter.print(). ■ 7. PrinterAdapter.print() executes: ■
legacyPrinter.printDocument(); ■ → Calls printDocument() on the stored LegacyPrinter
reference. ■ 8. LegacyPrinter.printDocument() executes: ■ System.out.println("Legacy
Printer"); ■ → Prints: "Legacy Printer" ■ 9. Control unwinds: printDocument() returns,
print() returns, main() returns. JVM exits with code 0.
```

The key observation: the call chain `print() → printDocument()` is invisible to the client. At step 6 the client believes it is talking to a generic `Printer`; the two delegation steps (6 → 7 → 8) are entirely encapsulated inside the Adapter.

Real-World Use Cases

1. Payment Gateway Integration

An e-commerce platform is built against an internal `PaymentGateway` interface with a method `processPayment(Order order)`. The company later integrates Stripe, whose SDK exposes `StripeClient.charge(ChargeParams params)`. Rather than rewriting all payment code to use Stripe's API, a `StripePaymentAdapter` wraps `StripeClient`, implements `PaymentGateway`, converts an `Order` into `ChargeParams`, and delegates. If the company later switches to Braintree, only a new `BraintreePaymentAdapter` is written; the rest of the application is unchanged.

```
public class StripePaymentAdapter implements PaymentGateway { private final StripeClient stripeClient; public StripePaymentAdapter(StripeClient stripeClient) { this.stripeClient = stripeClient; } @Override public PaymentResult processPayment(Order order) { ChargeParams params = ChargeParams.builder() .amount(order.getTotalCents()) .currency(order.getCurrencyCode()) .source(order.getPaymentToken()) .build(); Charge charge = stripeClient.charge(params); return new PaymentResult(charge.getId(), charge.getStatus()); } }
```

2. AWS SDK Wrapping

Applications that use the AWS SDK directly become tightly coupled to `AmazonS3` or `S3Client` types. Introducing an `ObjectStorageAdapter` that implements a simple `ObjectStore` interface (with `upload`, `download`, `delete`) decouples the business logic from AWS. The same application can later be adapted to use Google Cloud Storage or a local filesystem for integration testing — the only change is which adapter is wired in.

```
public class S3ObjectStoreAdapter implements ObjectStore { private final S3Client s3Client; private final String bucketName; @Override public void upload(String key, byte[] data) { s3Client.putObject(PutObjectRequest.builder() .bucket(bucketName).key(key).build(), RequestBody.fromBytes(data)); } }
```

3. Jackson `ObjectMapper` with Legacy Serializers

Pre-Jackson code often used a custom `LegacyJsonSerializer` with a `toJson(Object obj)` method. When migrating to Jackson's `ObjectMapper`, a `JacksonSerializerAdapter` can implement the legacy `JsonSerializer` interface while delegating to `ObjectMapper.writeValueAsString()`. The migration is invisible to the rest of the codebase.

```
public class JacksonSerializerAdapter implements JsonSerializer { private final
ObjectMapper objectMapper; @Override public String toJson(Object obj) { try { return
objectMapper.writeValueAsString(obj); } catch (JsonProcessingException e) { throw new
SerializationException(e); } } }
```

4. Spring's `HandlerAdapter`

Spring MVC internally uses the Adapter pattern to support multiple controller styles. The `DispatcherServlet` works against a `HandlerAdapter` interface. `RequestMappingHandlerAdapter` adapts `@RequestMapping`-annotated methods; `HttpRequestHandlerAdapter` adapts `HttpRequestHandler` implementations; `SimpleControllerHandlerAdapter` adapts the older `Controller` interface. The `DispatcherServlet` (the client) calls `HandlerAdapter.handle(request, response, handler)` without knowing which controller style is in use. Each `HandlerAdapter` implementation wraps a different controller type and translates the generic `handle()` call into the specific invocation that the underlying controller expects.

5. JDBC Driver

The JDBC specification defines the `java.sql.Driver` interface. Every database vendor (MySQL, PostgreSQL, Oracle) ships its own adapter that implements this interface and delegates to the vendor's proprietary wire protocol. Application code calls `DriverManager.getConnection(url)` and receives a `Connection` — a standard interface. The MySQL driver's internal classes translate every `Statement.execute(sql)` call into MySQL's binary protocol packets. Switching databases is, in principle, a matter of swapping the JDBC driver jar and the connection URL.

Class Adapter vs Object Adapter

The GoF book describes two structural variants of the Adapter pattern. This codebase implements the **Object Adapter**.

Object Adapter (this implementation)

The Adapter holds a **reference** (a field) to the Adaptee and delegates calls to it. The relationship is **composition** ("has-a").

```
// Object Adapter – composition public class PrinterAdapter implements Printer { private
final LegacyPrinter legacyPrinter; // <-- field reference public
PrinterAdapter(LegacyPrinter legacyPrinter) { this.legacyPrinter = legacyPrinter; }
@Override public void print() { legacyPrinter.printDocument(); // <-- delegation } }
```

Advantages of Object Adapter:

- Works with the Adaptee and all of its subclasses — you can pass any subclass of `LegacyPrinter`

to the constructor.

- The Adaptee does not need to be known at compile time if you program to an interface.
- Preferred in Java because Java does not support multiple class inheritance.
- The Adaptee can be swapped at runtime by passing a different instance to the constructor.

Class Adapter (hypothetical Java approximation)

The Adapter **inherits** from the Adaptee (or uses multiple inheritance in C++). In Java, this is only partially achievable: the Adapter can `extend` the Adaptee and `implement` the Target interface.

```
// Class Adapter – inheritance (hypothetical Java version) public class
PrinterClassAdapter extends LegacyPrinter implements Printer { @Override public void
print() { printDocument(); // calls the inherited method from LegacyPrinter } }
```

Drawbacks of Class Adapter in Java:

- Binds the Adapter to one specific Adaptee class — subclasses of `LegacyPrinter` are not automatically covered.
- Java allows only single class inheritance, so if `LegacyPrinter` already extends something else, the Adapter cannot also extend a second useful class.
- The Adapter exposes the Adaptee's `public` methods directly, leaking the legacy API through the adapter.
- This form is generally discouraged in Java. C++ uses it more often because it supports true multiple inheritance.

Verdict: For Java, always prefer the Object Adapter (composition). The Class Adapter is a valid option in C++ and in languages that support mixins.

Key Design Decisions

`private final LegacyPrinter legacyPrinter` — Why `private` and `final`?

`private` enforces encapsulation: the Adaptee is an implementation detail of the Adapter. Nothing outside `PrinterAdapter` should know or care that a `LegacyPrinter` exists. If the field were `protected` or `public`, code elsewhere could bypass the Adapter and call `printDocument()` directly, defeating the purpose of the pattern.

`final` enforces immutability of the collaborator reference. Once the Adapter is constructed, its Adaptee cannot be replaced. This eliminates an entire class of bugs (e.g., a race condition where the Adaptee is swapped between calls in a multi-threaded environment). It also communicates intent: this Adapter is designed for exactly one Adaptee instance over its lifetime.

Constructor Injection — Why not a setter?

The Adaptee is a **required** dependency. An Adapter without an Adaptee is meaningless and would throw a `NullPointerException` the moment `print()` is called. Requiring the Adaptee in the constructor makes invalid state — a constructed `PrinterAdapter` with no `LegacyPrinter` — impossible to represent. Setter injection would allow `new PrinterAdapter()` to succeed, deferring the failure to the first method call, which is harder to diagnose.

Constructor injection also plays well with Spring's `@Autowired` constructor injection, making `PrinterAdapter` a valid Spring bean without any framework-specific annotations in the field.

`implements Printer` — Why not extends some base class?

Implementing an interface rather than extending a base class keeps the Adapter's inheritance slot free. If `LegacyPrinter` were already a subclass of some framework base class, a class-based Adapter would be impossible in Java. Using `implements` means the Adapter can extend any class it needs to for other reasons while still satisfying the `Printer` contract.

Interfaces also carry no hidden state, which means the Adapter's behaviour is entirely determined by its own fields and the Adaptee's behaviour — there are no surprise inherited fields or methods to reason about.

Separate Packages — Why not one package?

A single package would work at compile time, but it would conceal the structural roles of each class. The three-package layout (`target`, `adaptee`, `adapter`) makes the dependency graph visible in the filesystem. A new team member can infer from the package names alone that `target` is the contract, `adaptee` is the legacy code, and `adapter` is the bridge. It also prepares the code for realistic scenarios where `adaptee` is replaced by a Maven dependency (a `.jar`) and `target` is published as a shared API module.

Output

Running `AdapterDesignPattern.main()` produces:

```
Adapter Design Pattern Legacy Printer
```

Line 1 comes from the header `System.out.println` in `main()`. Line 2 comes from `LegacyPrinter.printDocument()`, invoked through the adapter chain `printer.print()` → `PrinterAdapter.print()` → `legacyPrinter.printDocument()`.

Summary

Role	Class	Package	Responsibility
Target	Printer	target	Defines the interface the client was written against
Adaptee	LegacyPrinter	adaptee	Holds the real behaviour; incompatible method name
Adapter	PrinterAdapter	adapter	Implements Target; delegates to Adaptee
Client	AdapterDesignPattern	root	Constructs the objects; uses only the Target interface

The Adapter pattern solves a fundamental software engineering problem: you have working code and a working interface, but they were not designed to fit together, and you cannot modify either. Rather than forcing a rewrite of the legacy code or a breaking change to the established interface, you write a thin translation layer — the Adapter — that makes them cooperate transparently. The pattern is ubiquitous in production Java: every JDBC driver, every Spring `HandlerAdapter`, and every cloud SDK wrapper you have ever used is an Adapter in disguise.